OpenFOAM and the H_2/CH_4 T-Junction Project — Self-Study Notes for the MTP Viva

Sakhare Yash Balram (2022CH71496)
Department of Chemical Engineering, IIT Delhi

April 2026

Abstract

This document is a single-stop reference for everything I need to be able to discuss in the viva: how OpenFOAM works as a CFD framework, what is in every file of my case directory, why each numerical and physical choice was made, and what the project pipeline looks like end-to-end. It is dense on purpose — it is meant to read like lecture notes, not a polished report. Equations and code listings are shown in full where they would otherwise force me to wave my hands.

Contents

1 What OpenFOAM actually is **2**
1.1 The Finite Volume Method in one paragraph . . . 2
1.2 The OpenFOAM solver lifecycle . . . 3

2 Anatomy of an OpenFOAM case directory **4**

3 The `system/` directory — numerics and controls **5**
3.1 controlDict — time, output, diagnostics . . . 5
3.1.1 Things to know about controlDict . . . 5
3.2 fvSchemes — spatial and temporal schemes . . . 6
3.2.1 What "Gauss linear corrected" means . . . 7
3.3 fvSolution — linear solvers and PIMPLE . . . 7
3.3.1 What PIMPLE actually does, step by step . . . 8
3.4 blockMeshDict — the background mesh . . . 9
3.5 snappyHexMeshDict — carving the geometry . . . 9
3.5.1 Phase 1: castellation (refinement) . . . 10
3.5.2 Phase 2: snap and Phase 3: layer addition . . . 11
3.6 decomposeParDict — parallel decomposition . . . 11

4 The `constant/` directory — physical models **13**
4.1 turbulenceProperties (and momentumTransport) . . . 13
4.1.1 The k-ω SST model in 30 seconds . . . 13
4.2 thermophysicalProperties . . . 14
4.3 The other small files in constant/ . . . 14

5 The `0/` directory — initial and boundary conditions **15**
5.1 0/U — velocity (the most interesting one) . . . 15
5.1.1 Reading the codedFixedValue inline-C++ . . . 16
5.1.2 The 30° extension — tilted branch . . . 16
5.2 0/p_rgh — modified pressure . . . 17
5.2.1 What `fixedFluxPressure` and `prghPressure` mean . . . 17
5.3 0/H2 and 0/CH4 — species mass fractions . . . 17
5.4 0/k and 0/omega — turbulence inlet BCs . . . 18
5.5 0/nut, 0/alphat, 0/T, 0/p . . . 18

6 The Allrun pipeline — what actually runs **20**
6.0.1 The eight things this does . . . 20

7 The DoE pipeline — how the 10 cases are generated **22**
7.1 lhs_design.py — the sliced LHS sampler . . . 22
7.2 stamp_cases.py — materialise the 10 case dirs . . . 22
7.3 run_doe.sh — the orchestrator . . . 22
7.4 generateSTL.py — the geometry . . . 23

8 Numerics refresher — the things you'll be asked **24**
8.1 Why the time scheme is Euler and not Crank–Nicolson . . . 24
8.2 Why upwind and not vanLeer / linearUpwind . . . 24
8.3 What CFL / Courant number actually controls . . . 24
8.4 What `p` vs `p_rgh` actually buys you . . . 24

9 Physics refresher — the dimensionless numbers **25**
9.1 What the project's primary metrics are, mathematically . . . 25

10 Things I might be asked, with crisp answers **26**

11 Quick command-line cheatsheet **29**

12 Final pre-viva checklist **30**

1. What OpenFOAM actually is

OpenFOAM (*Open-source Field Operation And Manipulation*) is a C++ library and a command-line toolbox for solving systems of PDEs on unstructured meshes by the **Finite Volume Method (FVM)**. It is not a single black-box solver: it is a collection of ~ 100 pre-built executables (mesh utilities, solvers, post-processors) plus a library you can extend.

The three things to remember:

1. **It is a library, not a GUI.** Every case is a directory of plain-text dictionaries. There is no "project file". You set up the case by writing dictionaries; you run the case by calling executables in sequence; you visualise results in ParaView (or by post-processing scripts).

2. **It uses the Finite Volume Method.** The PDEs (Navier–Stokes, species transport, k, ω, h, $\rho\ldots$) are integrated over each finite cell; surface fluxes are reconstructed from cell-centre values via interpolation schemes; boundary conditions are applied face-by-face.

3. **Solvers are application-specific.** Different physics gets different solvers — e.g. `simpleFoam` (steady incompressible), `pimpleFoam` (transient incompressible), `rhoPimpleFoam` (transient compressible), `rhoReactingFoam` (compressible with species), and `rhoReactingBuoyantFoam` (compressible + species + gravity, which is the one we use).

Why OpenFOAM and not ANSYS Fluent?

Three reasons that hold up in viva:

1. *Cost / licensing.* A Fluent licence is not feasible to keep alive on a personal workstation for a year-long DoE; OpenFOAM is free and runs on Linux/WSL out of the box.
2. *Reproducibility.* The case is plain text and lives in git; the entire DoE can be regenerated from a seed and a stamp script. With Fluent the journal/.cas binary is hard to diff or review.
3. *Extensibility.* Custom boundary conditions (`codedFixedValue` 1/7-th power-law, our angle-tilted branch profile) drop straight into the case as inline C++ snippets, no UDF compilation step.

1.1. The Finite Volume Method in one paragraph

For each conservation law $\partial_t\phi + \nabla\cdot\mathbf{F}(\phi) = S(\phi)$, the FVM integrates over a control volume V_P and uses the divergence theorem:

$$\frac{d}{dt}\int_{V_P} \phi\, dV \;+\; \oint_{\partial V_P} \mathbf{F}\cdot\hat{\mathbf{n}}\, dA \;=\; \int_{V_P} S\, dV. \tag{1}$$

The cell-centred ϕ_P is the unknown; surface integrals are discretised by summing over faces f of P:

$$V_P \frac{\phi_P^{n+1} - \phi_P^n}{\Delta t} \;+\; \sum_{f\in\partial V_P} \mathbf{F}_f\cdot\mathbf{S}_f \;=\; V_P S_P. \tag{2}$$

The flux $\mathbf{F}_f$ at the face needs ϕ_f, which is interpolated from ϕ_P and ϕ_N (neighbour) by an *interpolation scheme*: linear, upwind, vanLeer, MUSCL… The choice of scheme is what `system/fvSchemes` controls.

1.2. The OpenFOAM solver lifecycle

Every CFD case in OpenFOAM goes through the same sequence:

1. **Geometry.** Either generate a mesh directly with `blockMesh` (block-structured) or build it from STLs with `snappyHexMesh`.
2. **Initial and boundary conditions.** Edit the dictionaries in `0/` (one per field).
3. **Numerics.** Edit `system/fvSchemes` (spatial schemes), `system/fvSolution` (linear solvers, pressure-velocity coupling), `system/controlDict` (time stepping, write controls, function objects).
4. **Decomposition.** For parallel runs, `decomposePar` splits the mesh into N sub-meshes (`processor0/`, `processor1/`, ...).
5. **Solve.** `mpirun -np N solver -parallel` runs the actual transient.
6. **Reconstruct.** `reconstructPar` merges the decomposed time directories back into a single `0.1/`, `0.2/`, ..., on the host process.
7. **Post-process.** ParaView, function objects (sampled planes, surface integrals), or external scripts for derived quantities.

2. Anatomy of an OpenFOAM case directory

Every OpenFOAM case — ours included — has exactly three top-level directories, and that's the contract.

```
case_03/
|
+-- 0/                      <-- initial + boundary conditions per field
|   |-- U                       velocity
|   |-- p                       absolute pressure
|   |-- p_rgh                   modified pressure (= p - rho*g*x)
|   |-- T                       temperature
|   |-- H2, CH4                 species mass fractions
|   |-- k, omega                turbulence quantities
|   `-- nut, alphat             eddy-viscosity / -diffusivity
|
+-- constant/               <-- physical models (do not change with time)
|   |-- polyMesh/               the actual mesh (created by snappy/blockMesh)
|   |-- triSurface/             the STL geometry  (input to snappyHexMesh)
|   |-- thermophysicalProperties  thermo + species + transport
|   |-- turbulenceProperties      RANS model selection
|   |-- momentumTransport         alias of the above (forward-compat)
|   |-- reactions                 (empty -- no combustion)
|   |-- thermo.compressibleGas    (empty placeholder)
|   |-- combustionProperties      "off"
|   `-- g                        gravitational acceleration vector
|
`-- system/                 <-- "how to solve it"
    |-- controlDict          time-stepping, write controls, function objects
    |-- fvSchemes            ddt / div / grad / laplacian schemes
    |-- fvSolution           linear solvers + PIMPLE settings
    |-- blockMeshDict        background mesh definition
    |-- snappyHexMeshDict    snapped-Cartesian-cut-cell controls
    |-- decomposeParDict     parallel domain decomposition
    |-- createPatchDict      patch renaming / merging
    |-- surfaceFeatureExtractDict   feature edge extraction for snappy
    `-- postProcess.dict     post-processing recipes (optional)
```

Why three directories? The split mirrors the three independent decisions in any CFD problem: *what is the state at $t = 0$ and on the boundaries* (`0/`), *what physical model do we use* (`constant/`), and *what numerics do we use to solve it* (`system/`). They are physically separate so e.g. a mesh study can swap `constant/polyMesh/` without touching the BCs, and a schemes study can swap `system/fvSchemes` without touching the mesh.

3. The `system/` directory — numerics and controls

3.1. controlDict — time, output, diagnostics

`controlDict` controls *when* the solver runs, *how often* it writes, and *what runtime diagnostics* it computes. Annotated extract from our case 03:

```
application         rhoReactingBuoyantFoam;   // <-- which solver to use

startFrom           startTime;                // start from t = startTime
startTime           0;                        //   (latestTime to resume)
stopAt              endTime;                  // stop at t = endTime
endTime             1.2;                      // 1.2 s = ~2 flow-throughs
deltaT              1e-5;                     // initial dt; will adapt

writeControl        adjustable;               // write at runTime intervals
writeInterval       0.1;                      // write every 0.1 s of sim time
purgeWrite          3;                        // keep last 3 time-dirs
writePrecision      10;
runTimeModifiable   true;                     // re-read controlDict on the fly

adjustTimeStep      yes;                      // CFL-based dt adaptation
maxCo               4.5;                      // CFL cap
maxDeltaT           0.01;                     // hard cap on dt

functions
{
    residuals { type solverInfo; ... }       // per-step residual log
    yPlusAudit { type yPlus; ... }           // y+ stats every 0.1s
    mainInletFlux  { type surfaceFieldValue; fields (phi); ... }
    branchInletFlux{ type surfaceFieldValue; fields (phi); ... }
    outletFlux     { type surfaceFieldValue; fields (phi); ... }
    H2_outletAvg   { type surfaceFieldValue; fields (H2); operation areaAverage; }
    p_rgh_inlet    { ...areaAverage of p_rgh on main_inlet... }
    p_rgh_outlet   { ...areaAverage of p_rgh on outlet... }
    plane_z3_H2    { type surfaceFieldValue; regionType sampledSurface;
                     sampledSurfaceDict { type plane; ... } }
    // similar for z=4,5,6
    fieldAverage   { type fieldAverage; timeStart 0.6;
                     fields ( U  H2  CH4  p_rgh  k ); }
}
```

Listing 1: `system/controlDict` (annotated)

3.1.1. Things to know about controlDict

- **`adjustTimeStep yes; maxCo 4.5`** : the solver automatically picks Δt at every step so that the maximum Courant number $\mathrm{Co} = |u|\,\Delta t/\Delta x$ across the mesh stays below 4.5. PIMPLE with two correctors can stably handle $\mathrm{Co} \sim 5$. We later raised this to 8.0 for the 30° DoE to halve wall time.

- **`writeInterval 0.1`** : the solver writes a full snapshot every 0.1 s of simulated time. Combined with `purgeWrite 3`, only the last 3 snapshots are kept on disk (otherwise a 1.2 s run would write 12 directories of ~ 30 MB each).

- **Function objects** run *inside the solver loop*, not as a post-processing pass. They are the right way to compute Δp, y^+, and area-averaged H_2 at every step without storing every snapshot.

- **fieldAverage** starts time-averaging at $t_{\text{start}} = 0.6\,\text{s}$, after the start-up transient has cleared (one main-pipe flow-through). `prime2Mean on` for H_2 also accumulates the second moment so we can compute $\sigma^2_{H_2}$.

3.2. fvSchemes — spatial and temporal schemes

`fvSchemes` tells the solver which numerical scheme to use for each derivative term in the equations.

```
ddtSchemes
{
    default         Euler;                          // 1st-order implicit
}

gradSchemes
{
    default         Gauss linear;
    grad(U)         cellLimited Gauss linear 1;     // limited to bound velocity
    grad(p_rgh)     Gauss linear;
    grad(H2)        cellLimited Gauss linear 1;
}

divSchemes
{
    default         none;                           // force every term to be set

    div(phi,U)      Gauss upwind;                   // 1st-order upwind, stable
    div(phi,H2)     Gauss upwind;
    div(phi,CH4)    Gauss upwind;
    div(phi,k)      Gauss upwind;
    div(phi,omega)  Gauss upwind;
    div(phi,h)      Gauss upwind;
    div(phi,K)      Gauss upwind;
    div(phi,Yi_h)   Gauss multivariateSelection { H2 upwind; CH4 upwind; h upwind; };
    div(phiv,p)     Gauss upwind;
    div(((rho*nuEff)*dev2(T(grad(U))))) Gauss linear;   // viscous stress
}

laplacianSchemes
{
    default         Gauss linear corrected;          // 2nd-order, snGrad-corrected
}

interpolationSchemes
{
    default         linear;                          // central interpolation
}

snGradSchemes
{
    default         corrected;                       // surface-normal grad with
        non-orthog correction
}

wallDist
{
    method          meshWave;                        // wall distance for SST
}

fluxRequired
{
    default         no;
```

```
    p_rgh;                                          // p_rgh flux is needed for momentum
}
```

Listing 2: `system/fvSchemes` (full)

Why first-order upwind for div?

Upwind is *numerically diffusive* (it artificially smears sharp gradients), but it is *monotonic* — it does not generate wiggles. For H_2 mass fraction this matters a lot because second- or higher-order schemes can overshoot beyond $[0, 1]$ in cells with sharp gradients (the jet edge), which the JANAF thermophysics will then reject as unphysical. The cost is $\sim$ 5–10% excess diffusion in CoV (Ferziger & Perić 2002, sec. 8.8), which is acceptable for a parametric DoE where we look at *trends* not absolute numbers. Compare with literature: Su et al. also use first/second-order upwind for k, ε and energy.

3.2.1. What "Gauss linear corrected" means

- **Gauss**: surface-integral form (the default for FVM).
- **linear**: linear interpolation from cell centres to faces (i.e. $\phi_f = w\phi_P + (1 - w)\phi_N$).
- **corrected**: the surface-normal gradient at f is corrected for mesh non-orthogonality (i.e. when P, N, and the face centre are not collinear). This is what prevents second-order schemes from breaking down on the kind of polyhedral cells that snappyHexMesh produces.

3.3. fvSolution — linear solvers and PIMPLE

`fvSolution` is a two-part dictionary:

1. `solvers {...}` : how to solve the linear system associated with each field.
2. `PIMPLE {...}` : how the pressure–velocity coupling loop is structured.

```
solvers
{
    "rho.*"     { solver diagonal; }                    // density: trivial diagonal solve

    p           { solver GAMG; smoother GaussSeidel;    // GAMG = geometric algebraic multigrid
                  tolerance 1e-7; relTol 0.01; }

    p_rgh       { solver PCG; preconditioner DIC;       // PCG = preconditioned conjugate gradient
                  tolerance 1e-7; relTol 0.01; maxIter 200; }
    p_rghFinal  { $p_rgh; relTol 0; }                   // last corrector: solve to convergence

    "U"         { solver smoothSolver; smoother symGaussSeidel;
                  tolerance 1e-7; relTol 0.1; nSweeps 2; }
    UFinal      { $U; relTol 0; }

    "(H2|CH4|Yi|k|omega|epsilon)"
                { solver PBiCGStab; preconditioner DILU;
                  tolerance 1e-7; relTol 0.1; }
```

```
    "(H2|CH4|Yi|k|omega|epsilon)Final" { $H2; relTol 0; }
}

PIMPLE
{
    momentumPredictor         yes;      // step (a): predict U from old p
    nOuterCorrectors          1;        // # of full PISO outer loops per dt
    nCorrectors               2;        // # of pressure corrections per outer
    nNonOrthogonalCorrectors  1;        // # of mesh-non-ortho corrections
    transonic                 no;
}

relaxationFactors
{
    equations { "U.*" 0.7; "k.*" 0.7; "omega.*" 0.7;
                "H2.*" 0.9; "CH4.*" 0.9; }
    fields    { "p_rgh.*" 0.3; }
}
```

Listing 3: `system/fvSolution` (annotated extract)

Why PCG/DIC for p_{rgh} instead of GAMG?

GAMG (geometric–algebraic multigrid) is normally faster, but on compressible flows with steep density contrasts at the junction, it sometimes triggers a floating-point exception in the agglomeration step (the coarse-level coefficients become ill-conditioned). PCG with DIC preconditioner is slower per iteration but rock-solid; for our $\sim 460\,\mathrm{k}$ cell mesh it converges in $\lesssim 50$ iterations, which is fine.

3.3.1. What PIMPLE actually does, step by step

PIMPLE = *merged PISO + SIMPLE*. For each timestep:

1. **Momentum predictor** (if `momentumPredictor yes`): solve momentum with old p, get a tentative $\mathbf{u}^*$.
2. **Pressure correction** (PISO loop, repeated `nCorrectors` times):
   - Build the pressure equation from continuity using $\mathbf{u}^*$.
   - Solve p_{rgh}.
   - Correct $\mathbf{u}$ using the new pressure gradient.
3. **Repeat** (SIMPLE outer loop, `nOuterCorrectors` times) if outer-loop convergence is needed within the timestep.
4. **Solve scalars**: k, ω, h, H_2.
5. **Update** ρ, μ, ν_t from thermo and turbulence model.

For our `nOuterCorrectors=1, nCorrectors=2`, that is "one outer iteration with two PISO corrections per timestep" — the PISO mode of PIMPLE, suitable for transient flows where the time step is small enough ($\mathrm{Co} \leq 5$).

Likely viva question

"Why one outer corrector and not three?"
Because we are doing a *transient* simulation with a small time step (Co ≤ 4.5). In transient mode, the field changes between time steps are small, so a single outer iteration plus two PISO inner corrections is enough to drop the residuals by 5–6 orders of magnitude per step. More outer iterations would just multiply wall time without improving accuracy — they only help when you push Co $\gg 1$ to chase a steady state.

3.4. blockMeshDict — the background mesh

`blockMesh` reads `system/blockMeshDict` and emits a structured hex mesh of the bounding box. This is the *background* mesh that snappyHexMesh later carves the geometry out of.

```
scale   1;

xMin 0.00; xMax 0.30;      // half-domain (x = 0 is symmetry plane)
yMin -0.30; yMax 1.50;
zMin -0.05; zMax 6.95;     // R2 shortened upstream length (was up to z=-2.3)

vertices ( ($xMin $yMin $zMin) ... ($xMin $yMax $zMax) );  // 8 corners

blocks
(
    hex (0 1 2 3 4 5 6 7) (10 60 232) simpleGrading (1 1 1)
);
//                          ^^^^^^^^^^^^
//   uniform 30 mm cells in all 3 directions = 139,200 background cells

boundary
(
    sym
    {
        type symmetryPlane;             // x = 0 face is the mirror plane
        faces ( (0 4 7 3) );
    }
    allBoundary
    {
        type patch;                     // catch-all for snappy to retag
        faces ( ... five other faces ... );
    }
);
```

Listing 4: `system/blockMeshDict` (key parts)

Two things to flag. First, `xMin = 0` means we run only the half-domain $x \geq 0$ and recover the other half by mirror symmetry, halving the cell count. Second, `allBoundary` is a deliberate catch-all for the other five faces of the bounding box — snappyHexMesh will later split it into the actual `wall`, `main_inlet`, `outlet` and `branch_inlet` patches based on the STL files.

3.5. snappyHexMeshDict — carving the geometry

snappyHexMesh has three phases controlled by three flags at the top:

```
castellatedMesh true;      // (1) hex-cut cells where they intersect STLs
snap            true;      // (2) snap cell points to STL surface
addLayers       true;      // (3) extrude prismatic layers along walls
```

3.5.1. Phase 1: castellation (refinement)

```
geometry
{
    wall.stl          { type triSurfaceMesh; name wall; }
    main_inlet.stl    { type triSurfaceMesh; name main_inlet; }
    outlet.stl        { type triSurfaceMesh; name outlet; }
    branch_inlet.stl  { type triSurfaceMesh; name branch_inlet; }

    // Sphere of refinement around the junction
    junctionRefine
    {
        type        searchableSphere;
        centre      (0 0 2.3);                  // junction centre
        radius      0.69;                       // 1.5*D_main
    }
    // Cylindrical refinement extending downstream of junction
    jetRefine
    {
        type        searchableCylinder;
        point1      (0 0 2.6);                  // just past junction
        point2      (0 0 6.2);                  // ~8 D_main downstream
        radius      0.23;                       // R_main
    }
}

castellatedMeshControls
{
    maxLocalCells       200000;
    maxGlobalCells      1500000;
    minRefinementCells  10;
    nCellsBetweenLevels 3;

    features                                    // edge refinement from .eMesh files
    (
        { file "wall.eMesh";         level 1; }
        { file "main_inlet.eMesh";   level 0; }
        { file "outlet.eMesh";       level 0; }
        { file "branch_inlet.eMesh"; level 1; }
    );

    refinementSurfaces
    {
        wall          { level (1 1); patchInfo { type wall; } }   // 1 level of surface refinement
        main_inlet    { level (0 0); patchInfo { type patch; } }
        outlet        { level (0 0); patchInfo { type patch; } }
        branch_inlet  { level (1 1); patchInfo { type patch; } }
    }

    resolveFeatureAngle 30;

    refinementRegions
    {
        junctionRefine { mode inside; levels ((1e15 2)); }   // level 2 inside sphere
        jetRefine      { mode inside; levels ((1e15 1)); }   // level 1 inside cylinder
```

```
    }

    locationInMesh (0.077 0.011 1.111);    // any point physically inside the fluid
    allowFreeStandingZoneFaces true;
}
```

Listing 5: `snappyHexMeshDict` – castellation

The way to read `level (1 1)` : a refinement level L means the cell side length is $\frac{1}{2^L}$ of the background cell. Our background is 30 mm, so `level 1` $\rightarrow$ 15 mm, and the inside of the junction sphere at `level 2` $\rightarrow$ 7.5 mm.

3.5.2. Phase 2: snap and Phase 3: layer addition

```
snapControls
{
    nSmoothPatch            3;
    tolerance               2.0;
    nSolveIter              50;
    nRelaxIter              5;
    nFeatureSnapIter        10;
    explicitFeatureSnap     true;
}

addLayersControls
{
    relativeSizes        true;          // layer thickness as fraction of adjacent cell
    layers
    {
        wall { nSurfaceLayers 3; }      // 3 prism layers on every wall face
    }
    expansionRatio       1.2;           // each layer 1.2x thicker than the one below
    finalLayerThickness  0.5;           // outermost layer = 0.5 of adjacent cell
    minThickness         0.15;          // give up if a layer would be < 15%
    nLayerIter           40;
}
```

The *snap* phase moves cell points onto the STL surface so the mesh actually conforms to the geometry. The *addLayers* phase then extrudes 3 prismatic layers from the wall inward to land $y^+ \sim 30$–80 at the first cell, which is the wall-function regime our k–ω SST setup needs.

Likely viva question

"Why $y^+ \sim 30$–80 instead of resolving to $y^+ \sim 1$?"
With $y^+ \sim 1$ we would need $\sim 5\times$ more cells in the boundary layer and the time step would have to drop by a similar factor, since the smallest cell sets the CFL limit. For a parametric DoE this is unaffordable. Wall functions (`kqRWallFunction`, `omegaWallFunction`) bridge the viscous sublayer analytically, which is industry standard for the Reynolds numbers we have ($Re \sim 5 \times 10^5$) and the quantities we report (CoV, Δp). For LES or heat-transfer studies this trade-off would not be acceptable.

3.6. decomposeParDict — parallel decomposition

```
numberOfSubdomains  16;
method              scotch;
```

Listing 6: `system/decomposeParDict` (full)

`scotch` is a graph-partitioning library that minimises the inter-process communication surface. The mesh is split into 16 sub-meshes (16 MPI ranks on the 8-core/16-thread workstation), each running a copy of the solver on its own `processor*/` folder. Each rank only needs values from neighbour ranks at sub-domain boundary faces (the "halo"), which is what scotch tries to keep small.

4. The `constant/` directory — physical models

4.1. turbulenceProperties (and momentumTransport)

```
simulationType  RAS;        // RAS = Reynolds-Averaged Simulation (RANS)

RAS
{
    RASModel        kOmegaSST;
    turbulence      on;
    printCoeffs     on;     // dump model constants to log on first call
}

Sct      0.5;               // Turbulent Schmidt number for species transport
kMin     1e-10;             // floor on k to avoid divide-by-zero
omegaMin 1e-10;
```

Listing 7: `constant/turbulenceProperties` (full)

Schmidt number $Sc_t = 0.5$. This sets the turbulent diffusivity for species ($D_{\text{turb}} = \nu_t/Sc_t$). The textbook range is 0.7–1.0 for inert gases; 0.5 is the calibrated value for high-density-ratio mixing (Su 2022, He et al. 2019).

4.1.1. The k-ω SST model in 30 seconds

- Solves two transport equations: one for k (turbulent kinetic energy) and one for ω (specific dissipation rate $\omega = \varepsilon/k$).
- Eddy viscosity: $\mu_t = \rho\, a_1\, k/\max(a_1\,\omega,\, F_2 S)$ where $a_1 = 0.31$, F_2 is a blending function active near walls, S is strain-rate magnitude. The $\max(\cdot)$ is the *shear-stress transport* (SST) limiter.
- Blending function F_1 switches between standard k–ω near walls (where it's good) and standard k–ε in the free shear (where it's better).
- Wall functions in our case (`kqRWallFunction`/`omegaWallFunction`) bridge the viscous sublayer analytically when $y^+ > 11$.

Why SST and not standard k-ε (which Su 2022 used)?

1. **Adverse pressure gradients.** k–ε over-predicts eddy viscosity in adverse ∇p regions (the lee side of the branch jet). SST limits this through F_2.
2. **Separation prediction.** k–ε under-predicts separation; SST handles separation length within $\sim 5\%$.
3. **Wall behaviour.** k–ε needs damping functions or a two-layer model near walls; SST does not.

The penalty is solving two equations whose coupling is more sensitive to discretisation. We pay that with cell-limited gradients in `fvSchemes` and PBiCGStab+DILU on k, ω.

4.2. thermophysicalProperties

This is where multispecies thermo lives.

```
thermoType
{
    type            heRhoThermo;        // density-based, sensible enthalpy
    mixture         reactingMixture;    // multi-species mixture
    transport       sutherland;         // Sutherland's law for mu(T)
    thermo          janaf;              // JANAF polynomials for cp(T)
    energy          sensibleEnthalpy;   // h_sensible = integral(cp dT)
    equationOfState perfectGas;         // p = rho R T
    specie          specie;
}

inertSpecie      CH4;                  // not solved for; closure species
species ( H2 CH4 );

H2
{
    specie { molWeight 2.016; }        // kg/kmol
    thermodynamics
    {
        Tlow 200; Thigh 6000; Tcommon 1000;
        highCpCoeffs ( 2.93287 8.27e-04 -1.46e-07 1.54e-11 -6.89e-16 -813.07 -1.024 );
        lowCpCoeffs  ( 2.34433 7.98e-03 -1.95e-05 2.02e-08 -7.38e-12 -917.94 0.683 );
    }
    transport { As 6.69e-07; Ts 72.0; }   // Sutherland mu = As * sqrt(T) / (1 + Ts/T)
}

CH4
{
    specie { molWeight 16.043; }
    thermodynamics { ... 7-coeff JANAF polynomial ... }
    transport      { As 1.17e-06; Ts 164.0; }
}
```

Listing 8: `constant/thermophysicalProperties` (key parts)

What `heRhoThermo` buys us. `heRhoThermo` is a density-based thermodynamics class: ρ is computed from p, T and the local mixture composition, and the energy variable is sensible enthalpy $h = \sum_k Y_k \int_{T_{\text{ref}}}^{T} c_{p,k}(T')\,dT'$. JANAF polynomials provide $c_p(T)$ as a 7-coefficient piecewise polynomial split at $T_{\text{common}} = 1000\,\text{K}$. This is what lets us treat H_2/CH_4 as a real binary mixture rather than two passive scalars.

4.3. The other small files in constant/

```
dimensions  [0 1 -2 0 0 0 0];
value       (0 -9.81 0);                    // gravity points -y (down)
```

Listing 9: `constant/g` (full)

This is what activates the buoyancy term in `rhoReactingBuoyantFoam`. `combustionProperties` is set to "off" (we don't want chemistry), `reactions` is empty, and `thermo.compressibleGas` is a placeholder demanded by the `foamChemistryReader` format.

5. The 0/ directory — initial and boundary conditions

There is one dictionary per field variable. The structure is always the same: `dimensions`, `internalField`, then a `boundaryField` block listing one entry per patch.

5.1. 0/U — velocity (the most interesting one)

This is where the *custom* 1/7-th power-law profile lives. Below is the full `codedFixedValue` for the main inlet and the branch inlet at $\alpha = 90°$ from case 03:

```
internalField   uniform (0 0 5);

boundaryField
{
    wall { type noSlip; }
    sym  { type symmetryPlane; }

    main_inlet
    {
        type            codedFixedValue;
        value           uniform (0 0 10);
        name            mainInletPowerLaw;
        code
        #{
            const vectorField& Cf = this->patch().Cf();    // face centres
            vectorField& field = *this;                    // the BC values
            const scalar d    = 0.46;                      // main pipe D
            const scalar uMax = 10;                        // main bulk velocity
            const scalar n    = 7.0;                       // 1/7-th law exponent
            forAll(Cf, i)
            {
                const scalar r = Foam::sqrt(sqr(Cf[i].x()) + sqr(Cf[i].y()));
                const scalar arg = Foam::max(1.0 - sqr(2.0*r/d), scalar(0));
                const scalar u   = uMax * Foam::pow(arg, 1.0/n);
                field[i] = vector(0, 0, u);
            }
        #};
    }

    outlet { type pressureInletOutletVelocity; value uniform (0 0 5); }

    branch_inlet
    {
        type            codedFixedValue;
        value           uniform (0 -13.3032 0);
        name            branchInletPowerLaw;
        code
        #{
            const vectorField& Cf = this->patch().Cf();
            vectorField& field = *this;
            const scalar d    = 0.11585;
            const scalar uMax = 13.3032;
            const scalar zc   = 2.3;                       // junction z
            const scalar n    = 7.0;
            forAll(Cf, i)
            {
                const scalar r = Foam::sqrt(sqr(Cf[i].x()) + sqr(Cf[i].z() - zc));
                const scalar arg = Foam::max(1.0 - sqr(2.0*r/d), scalar(0));
                const scalar u   = uMax * Foam::pow(arg, 1.0/n);
                field[i] = vector(0, -u, 0);               // injecting in -y
```

```
            }
        #};
    }
}
```

Listing 10: `0/U` – 90° version (case 03)

5.1.1. Reading the codedFixedValue inline-C++

- `this->patch().Cf()` returns the centre coordinate of every face on this patch.
- `vectorField& field = *this` : the BC array we are *writing* (per-face $\mathbf{u}$).
- For each face i, we compute the radial distance r from the pipe axis, plug it into the 1/7-th power law, $u(r) = u_{\max}(1 - (2r/d)^2)^{1/7}$, and write `field[i]`.
- `Foam::max(...,0)` guards against negative arguments at the wall (numerical safety) before the `Foam::pow`, which would otherwise return NaN.

5.1.2. The 30° extension — tilted branch

For an arbitrary angle, the branch axis is no longer aligned with $-y$, so we need to compute the radial distance *in the branch's local frame* and project the velocity vector along $-\hat{\mathbf{s}}$ where $\hat{\mathbf{s}}$ is the branch axis. This is the version stamped at 30° ($s_\alpha = \sin\alpha = 0.5$, $c_\alpha = \cos\alpha \approx 0.866$):

```
branch_inlet
{
    type            codedFixedValue;
    value           uniform (0 -6.2066 10.7501);   // = u_branch * (-sin a, cos a)
    name            branchInletPowerLaw;
    code
    #{
        const vectorField& Cf = this->patch().Cf();
        vectorField& field = *this;
        const scalar d    = 0.13631;
        const scalar uMax = 12.4132;
        const scalar zc   = 2.3;                   // junction z
        const scalar yJ   = 0.23;                  // junction y (= R_main)
        const scalar sa   = 0.5;                   // sin(alpha)
        const scalar ca   = 0.866025;              // cos(alpha)
        const scalar n    = 7.0;
        forAll(Cf, i)
        {
            // (1) vector from junction centre to face centre
            const scalar wx = Cf[i].x();
            const scalar wy = Cf[i].y() - yJ;
            const scalar wz = Cf[i].z() - zc;

            // (2) project onto the branch axis  S_HAT = (0, sa, -ca)
            const scalar sProj = wy*sa + wz*(-ca);

            // (3) perpendicular component (radial in the local frame)
            const scalar ppx = wx;
            const scalar ppy = wy - sProj*sa;
            const scalar ppz = wz - sProj*(-ca);
            const scalar r   = Foam::sqrt(ppx*ppx + ppy*ppy + ppz*ppz);

            // (4) 1/7-th law
```

```
            const scalar arg = Foam::max(1.0 - sqr(2.0*r/d), scalar(0));
            const scalar u   = uMax * Foam::pow(arg, 1.0/n);

            // (5) flow direction is -S_HAT = (0, -sa, ca)
            field[i] = vector(0.0, -u*sa, u*ca);
        }
    #};
}
```

Listing 11: `0/U` – 30° branch (the angle-agnostic version)

The point of the angle-agnostic code. With this construction, `stamp_cases.py` only needs to substitute the four scalars `(d, uMax, sa, ca)` into the template, and the same dictionary works for $\alpha = 30°$, $90°$, $150°$, or anything else. At $\alpha = 90°$ we have $s_\alpha = 1, c_\alpha = 0$, and the code reduces algebraically to the original "`vector(0, -u, 0)`" formulation.

5.2. 0/p_rgh — modified pressure

`p_rgh` is the modified pressure $p_{\text{rgh}} = p - \rho\mathbf{g}\cdot\mathbf{x}$ that lets us absorb the hydrostatic part out of the pressure equation. This is *the* field the buoyancy solver actually solves; `p` is reconstructed from p_{rgh} and $\rho\mathbf{g}\cdot\mathbf{x}$ at every step.

```
dimensions  [1 -1 -2 0 0 0 0];           // [Pa]
internalField   uniform 6900000;          // 6.9 MPa  (operating pressure of NG pipeline)

boundaryField
{
    main_inlet   { type fixedFluxPressure; value uniform 6900000; }
    branch_inlet { type fixedFluxPressure; value uniform 6900000; }
    outlet       { type prghPressure; p uniform 6900000; value uniform 6900000; }
    wall         { type fixedFluxPressure; value uniform 6900000; }
    sym          { type symmetryPlane; }
}
```

Listing 12: `0/p_rgh` (full)

5.2.1. What `fixedFluxPressure` and `prghPressure` mean

- `fixedFluxPressure` : the pressure gradient at the boundary is set so that the velocity-flux at the boundary is consistent with the actual mass flow there. It is the momentum-equation–consistent counterpart of the velocity BC — not a fixed value of pressure.
- `prghPressure` (on outlet) : sets the static pressure p at the outlet, then computes the corresponding p_{rgh} using the local density. So the user thinks in terms of p (intuitive), but the solver works with p_{rgh} (clean).

5.3. 0/H2 and 0/CH4 — species mass fractions

```
dimensions      [0 0 0 0 0 0 0];                // dimensionless
internalField   uniform 0;                       // start with zero H2 everywhere

boundaryField
{
    wall         { type zeroGradient; }
```

```
    sym          { type symmetryPlane; }
    main_inlet   { type fixedValue; value uniform 0; }   // pure CH4 enters from main
    outlet       { type inletOutlet; inletValue uniform 0; value uniform 0; }
    branch_inlet { type fixedValue; value uniform 1; }   // pure H2 enters from branch
}
```

Listing 13: `0/H2` (full)

The `inletOutlet` BC handles backflow at the outlet gracefully: if a face has $\mathbf{u}\cdot\hat{\mathbf{n}} > 0$ (outflow), it acts as zeroGradient; if it has back-flow, it injects `inletValue`. For our cases there is essentially no back-flow, but having this BC means a transient blip never crashes the run.

5.4. 0/k and 0/omega — turbulence inlet BCs

For incoming patches, the standard practice is to specify k from a turbulence intensity I and ω from a mixing length ℓ:

$$k = \tfrac{3}{2}(I\,U)^2, \qquad \omega = \frac{\sqrt{k}}{C_\mu^{1/4}\,\ell}, \qquad C_\mu = 0.09. \tag{3}$$

With $I = 5\%$ (typical for $Re > 10^5$ pipe flow), $U = 10\,\mathrm{m/s}$, $D = 0.46\,\mathrm{m}$, $\ell = 0.07\,D = 0.0322\,\mathrm{m}$, this gives $k_{\mathrm{main}} = 0.375\,\mathrm{m}^2/\mathrm{s}^2$ and $\omega_{\mathrm{main}} = 34.72\,1/\mathrm{s}$ — exactly the numbers in our case.

```
internalField uniform 0.375;

boundaryField
{
    main_inlet   { type turbulentIntensityKineticEnergyInlet; intensity 0.05; value
        uniform 0.375; }
    branch_inlet { type turbulentIntensityKineticEnergyInlet; intensity 0.05; value
        uniform 0.663661; }
    outlet       { type inletOutlet; inletValue uniform 0.375; value uniform 0.375; }
    wall         { type kqRWallFunction; value uniform 0.375; }
    sym          { type symmetryPlane; }
}
```

Listing 14: `0/k` (full)

```
internalField uniform 35;

boundaryField
{
    main_inlet   { type turbulentMixingLengthFrequencyInlet; mixingLength 0.0322;  value
        uniform 34.72; }
    branch_inlet { type turbulentMixingLengthFrequencyInlet; mixingLength 0.008109; value
        uniform 183.41; }
    outlet       { type inletOutlet; inletValue uniform 34.72; value uniform 34.72; }
    wall         { type omegaWallFunction; value uniform 34.72; }
    sym          { type symmetryPlane; }
}
```

Listing 15: `0/omega` (full)

5.5. 0/nut, 0/alphat, 0/T, 0/p

- `nut` : turbulent kinematic viscosity. The wall function uses `nutkWallFunction` to set ν_t at the wall from k and y^+. Internally OpenFOAM does this automatically; we just supply a non-zero

starting field.

- `alphat` : turbulent thermal diffusivity. Same story — a wall function field.
- `T` : temperature. We run isothermal at 300 K, so `T` is just a constant uniform 300 with all-walls `zeroGradient`; the energy equation is effectively deactivated (relTol 1, maxIter 0 in fvSolution).
- `p` : absolute pressure. Reconstructed from `p_rgh` every step; `0/p` is just an initial guess set to the same operating pressure as `0/p_rgh`.

6. The Allrun pipeline — what actually runs

The `Allrun` script ties everything together. Here is the real script for case 04 (annotated):

```bash
#!/usr/bin/env bash
set -eo pipefail
cd "$(dirname "$0")"

# 1. Source OpenFOAM environment
OPENFOAM_BASHRC="${OPENFOAM_BASHRC:-/usr/lib/openfoam/openfoam2406/etc/bashrc}"
set +eu
source "$OPENFOAM_BASHRC" 2>/dev/null
set -e

# 2. Per-case env (UMAIN, UBRANCH, D1, D2, ZJCT, ALPHA_DEG)
source ./case.env

# 3. If we have a same-d/D donor case, copy its mesh to skip ~30 min of meshing
if [[ "${REUSE_MESH_FROM:-}" != "" && -d "${REUSE_MESH_FROM}/constant/polyMesh" ]]; then
    cp -r "${REUSE_MESH_FROM}/constant/polyMesh"   constant/
    cp -r "${REUSE_MESH_FROM}/constant/triSurface" constant/ 2>/dev/null || true
    for p in "${REUSE_MESH_FROM}"/processor*; do
        dst="./$(basename "$p")"
        mkdir -p "$dst/constant"
        cp -r "$p/constant/polyMesh" "$dst/constant/"
    done
else
    # 4. Otherwise build mesh from scratch
    python3 generateSTL.py            | tee log.generateSTL
    python3 scripts/clip_stls.py constant/triSurface | tee log.clipStls
    surfaceFeatureExtract             | tee log.surfaceFeatureExtract
    blockMesh                          | tee log.blockMesh
    snappyHexMesh -overwrite           | tee log.snappyHexMesh
    if grep -q "allBoundary" constant/polyMesh/boundary; then
        createPatch -overwrite         | tee log.createPatch
    fi
    rm -f 0/cellLevel 0/pointLevel 0/cellZones_0 0/pointZones_0
    checkMesh -allTopology -allGeometry | tee log.checkMesh
    decomposePar -force                | tee log.decomposePar
fi

# 5. Refresh decomposed initial fields from the (possibly reused) mesh
rm -rf processor*/0 processor*/0.*
decomposePar -force | tee log.decomposePar

# 6. Run the actual solver in parallel
mpirun --use-hwthread-cpus -n 16 rhoReactingBuoyantFoam -parallel | tee log.solver

# 7. Reconstruct only the latest time directory back into the host
reconstructPar -latestTime | tee log.reconstructPar

# 8. Mark this case complete (run_doe.sh checks for SIM_DONE)
echo "[case_04] SIM DONE at $(date +%FT%T)"
touch SIM_DONE
```

Listing 16: `case_04/Allrun` – the per-case pipeline

6.0.1. The eight things this does

1. Source OpenFOAM (sets `$WM_PROJECT_DIR` etc.).

2. Read per-case parameters (`UMAIN, UBRANCH, D2, ...`).
3. Mesh-reuse shortcut: if the same-slice donor case has already meshed, just `cp` the mesh (saves ~ 30 minutes per case).
4. Otherwise, full mesh build:
   - `generateSTL.py` — analytic T-junction STL.
   - `clip_stls.py` — clip to half-domain $x \geq 0$.
   - `surfaceFeatureExtract` — extract sharp edges.
   - `blockMesh` — background mesh.
   - `snappyHexMesh -overwrite` — castellate, snap, layers.
   - `createPatch` — re-tag `allBoundary` faces.
   - `checkMesh` — mesh-quality report.
   - `decomposePar -force` — split into 16 ranks.
5. Refresh `processor*/0` from the source `0/` (ensures field sizes match a possibly reused mesh).
6. `mpirun -n 16 rhoReactingBuoyantFoam -parallel` — the actual transient.
7. `reconstructPar -latestTime` — merge the latest time directory.
8. `touch SIM_DONE` — the orchestrator sentinel.

7. The DoE pipeline — how the 10 cases are generated

7.1. lhs_design.py — the sliced LHS sampler

`lhs_design.py` produces `doe_design.csv`, a 10-row table of (d/D, HBR, VR) triplets:

```python
def sliced_lhs(seed=42, n_slices=5, m=2):
    rng = np.random.default_rng(seed)
    # 1. uniform LHS over (dD, HBR) box
    dD_levels  = np.linspace(0.15, 0.45, n_slices+1)
    cases = []
    for s, (lo, hi) in enumerate(zip(dD_levels[:-1], dD_levels[1:])):
        # 2. sample 2 HBR values within the slice
        hbr_lhs = lhs(2, m, seed=rng)
        for j in range(m):
            dD  = rng.uniform(lo, hi)
            hbr = 0.05 + 0.15 * hbr_lhs[j]
            # 3. invert HBR(dD, VR) = (dD)^2 * VR / (1 + (dD)^2 * VR)
            VR  = hbr / ((dD**2) * (1 - hbr))
            assert VR <= 12, "VR safety cap exceeded"
            cases.append((s+1, dD, hbr, VR))
    return cases
```

Listing 17: Sketch of `lhs_design.py`

The point of the slice structure. Each pair of cases shares the same d/D value (slice). Geometry only depends on d/D, so the same mesh works for both. The orchestrator `run_doe.sh` therefore builds a mesh once per slice (5 builds not 10), saving roughly 4 hours per campaign.

Why the soft cap on VR. HBR is bounded above at 0.20, but for very small d/D the equation $\mathrm{HBR} = (d/D)^2\mathrm{VR}/(1 + (d/D)^2\mathrm{VR})$ gives an enormous VR to hit 20% blend. A VR > 12 jet impinges on the opposite wall and erodes the pipe (and triggers very small time-steps in our solver). The soft cap is a physically motivated pruning of the parameter box.

7.2. stamp_cases.py — materialise the 10 case dirs

`stamp_cases.py` reads `doe_design.csv`, copies the `doe_base/` template, and replaces tokens like `@UBRANCH@`, `@D2@`, `@KBRANCH@`, `@SA@`, `@CA@` with the per-case numerics. After stamping, each `case_NN/` directory is a fully populated, ready-to-run OpenFOAM case (its own `0/`, `constant/`, `system/`, `Allrun`, `generateSTL.py`, `case_info.json`).

7.3. run_doe.sh — the orchestrator

The orchestrator iterates over the 10 cases, decides whether to build a mesh or reuse one, and handles per-case success/failure:

```bash
for cname in "${SELECTED[@]}"; do
    cdir="${CASES_DIR}/${cname}"
    [[ -d "${cdir}" ]] || continue

    if [[ -f "${cdir}/SIM_DONE" ]]; then
        log "[skip] ${cname}: already done"
        post_process "${cdir}"
        continue
```

```
    fi

    # mesh-reuse: find a donor in the same slice
    slice="$(case_slice_id "${cdir}")"
    donor="$(find_mesh_donor "${slice}" "${cdir}")" || true
    [[ -n "${donor}" ]] && export REUSE_MESH_FROM="${donor}" || unset REUSE_MESH_FROM

    if bash "${cdir}/Allrun" >> "${cdir}/log.run_doe" 2>&1; then
        log "[${cname}] SIM OK"
        stamp_wall_seconds "${cdir}/case_info.json" "${elapsed}" ok
        post_process "${cdir}"
    else
        log "[${cname}] SIM FAILED"
        tail -n 200 "${cdir}/log.run_doe" > "${cdir}/SIM_FAILED"
        post_process "${cdir}"
        log "[${cname}] continuing with next case"   # crash resilience
    fi
done
```

Listing 18: Core loop of `run_doe.sh`

Crash resilience. A failure in one case never aborts the campaign. The orchestrator copies whatever the case did manage to produce (the partial `processor*` state, the log) into `doe_results/case_NN/`, writes a `SIM_FAILED` sentinel with the last 200 log lines, and moves on. This is *exactly* how case 04 of the 30° campaign was paused on demand and the campaign continued on case 05.

7.4. generateSTL.py — the geometry

We do not draw the T-junction in a CAD package. Instead a Python script generates a watertight STL parametrically. The cylinders are sampled on a (θ, z) grid; the intersection curve between the branch cylinder and the main cylinder is computed analytically; a rectangular hole in the main pipe is punched in (θ, z) space; and the boundary of that hole is "zippered" to the intersection curve by triangulation. The output is a single STL with $\sim 44\,000$ triangles whose volume agrees with the analytic volume to better than 1%.

The 30° bug, in one sentence. The original code sized the rectangular hole using $z_{\text{stretch}} = R_2 + 0.5\, L_{\text{branch}}\, |\cos\alpha|$, which is fine at $\alpha = 90°$ ($\cos\alpha = 0$ collapses the heuristic), but at $\alpha = 30°$ punches a rectangle $\sim 7\times$ wider than the actual intersection curve, forcing the zipper to span $\sim 1\,\text{m}$ of empty annular space with stretched triangles that snappyHexMesh interpreted as a wall. The fix is to size the rectangle directly from the bbox of the intersection curve plus a few grid cells of buffer.

8. Numerics refresher — the things you'll be asked

8.1. Why the time scheme is Euler and not Crank–Nicolson

- **Euler (1st order, implicit):** stable, monotonic, cheap. Diffusive in time — but the time-averaging window (0.6–1.2 s) absorbs the diffusion, so reported QoIs are not affected.
- **Crank–Nicolson (2nd order):** less diffusive but can produce wiggles in stiff equations (energy, ω). For a parametric DoE the bookkeeping cost outweighs the benefit.
- **Backward (2nd order, BDF):** a compromise; we did not need it because the time-averaged QoIs converge fine with Euler.

8.2. Why upwind and not vanLeer / linearUpwind

- `upwind`: 1st order, monotonic, $O(\Delta x)$ diffusive.
- `linearUpwind`: 2nd order in smooth regions, can wiggle near sharp gradients.
- `vanLeer`: TVD limited 2nd order, bounded but noticeably more expensive per step.

For this DoE, upwind on $H_2/U/k/\omega$ keeps everything bounded and cheap; the $\sim$ 5–10% excess CoV from numerical diffusion is reported transparently (and is the same across all cases, so trends are unaffected).

8.3. What CFL / Courant number actually controls

The local Courant number is $\mathrm{Co} = |\mathbf{u}|\,\Delta t/\Delta x$. It measures the fraction of a cell a fluid parcel crosses in one time step. For explicit schemes the stability limit is $\mathrm{Co} \leq 1$. For implicit PIMPLE we can run $\mathrm{Co} \sim 5$–10 as long as the PIMPLE inner loop converges per step. Hence `maxCo 4.5` (90° campaign) and `maxCo 8.0` (30° campaign).

8.4. What `p` vs `p_rgh` actually buys you

If we kept solving for p directly, gravity would appear as a body force in the momentum equation, and the pressure equation would have to fight against the (large, easily-known) hydrostatic distribution $\rho\mathbf{g}\cdot\mathbf{x}$ to find the (small, interesting) kinematic part. By solving for $p_{\mathrm{rgh}} = p - \rho\mathbf{g}\cdot\mathbf{x}$ instead, the hydrostatic component drops out analytically and the solver only resolves the kinematic pressure — which is what matters for our Δp calculation. `p` is reconstructed post-hoc.

9. Physics refresher — the dimensionless numbers

Table 1: The four dimensionless numbers I should know cold.

Number	Definition	Meaning	Our regime
Reynolds	$Re = \rho U D/\mu$	inertia / viscous	5×10^5 (fully turbulent)
Courant	$\mathrm{Co} = \lvert\mathbf{u}\rvert \Delta t/\Delta x$	advection / cell time	$\leq$ 4.5–8.0 (numerical stability)
Richardson	$\mathrm{Ri} = g\Delta\rho L/(\rho U^2)$	buoyancy / shear	$\sim 10^{-2}$ (turbulence-dominated)
Momentum-flux ratio	$J = \rho_b u_b^2/(\rho_m u_m^2)$	jet penetration	0.7–160 across the DoE
Froude	$\mathrm{Fr} = U/\sqrt{g\Delta\rho L/\rho}$	inverse $\sqrt{\mathrm{Ri}}$	$\gg 1$ (production), ~ 0.3 (sanity)
Wobbe Index	$I_W = V_c/\sqrt{G}$	energy delivery per unit area	not modelled here

9.1. What the project's primary metrics are, mathematically

Coefficient of Variation (CoV) at outlet.

$$\mathrm{CoV} \;=\; \frac{\sigma_{Y_{H_2}}}{\overline{Y_{H_2}}} \;=\; \frac{\sqrt{\sum_i w_i (Y_{H_2,i} - \overline{Y_{H_2}})^2 / \sum_i w_i}}{\sum_i w_i Y_{H_2,i} / \sum_i w_i}. \tag{4}$$

Three weightings: $w_i = A_i$ (area), $w_i = \rho_i \, |\mathbf{u}\cdot\hat{\mathbf{n}}| A_i$ (mass-flux), $w_i = |\mathbf{u}\cdot\hat{\mathbf{n}}| A_i$ (volumetric-flux). The mass-flux weighting is the custody-transfer relevant one (because the customer downstream receives the mass-weighted average), but all three are reported. Industry threshold: CoV ≤ 0.05.

Static pressure drop. $|\Delta p| = |p_{\mathrm{main_inlet}} - p_{\mathrm{outlet}}|$, patch-area-averaged, time-averaged over the second flow-through.

Danckwerts segregation index.

$$I_s \;=\; \frac{\overline{(Y_{H_2} - \overline{Y_{H_2}})^2}}{\overline{Y_{H_2}} \cdot (1 - \overline{Y_{H_2}})}. \tag{5}$$

$I_s = 1$ means completely segregated (no mixing); $I_s = 0$ means perfectly mixed. Used as a cross-check on CoV because it is bounded.

10. Things I might be asked, with crisp answers

Q1: Why a T-junction and not a Y-junction or coaxial mixer?

T-junctions are the dominant industrial topology for hydrogen injection into existing natural-gas pipelines because they are *retrofittable*: a hot tap can punch the branch into a live main without taking it out of service. Y- and coaxial mixers require custom fabrication and a shutdown. The cost of poorer mixing is what we are quantifying.

Q2: Why three angles (90, 30, 150)?

90° is the industry baseline (cross-flow). 30° is co-flow — shallowest practical industrial geometry, gentle on the pipe. 150° is counter-flow — the opposite extreme. Three points span the regime; the linear/quadratic interpolation across angle gives you a usable design surface for any α in $[30°, 150°]$.

Q3: Why sliced LHS instead of full factorial?

A 3-factor 5-level full factorial is $5^3 = 125$ cases. We have budget for 10 per angle. LHS samples *uniformly* on every 1-D projection; sliced LHS additionally pre-stratifies d/D into 5 slices so we get mesh-reuse savings. Statistically, the effect of each factor is recovered to within $\sim 5\%$ of the full-factorial sensitivity for low-curvature responses.

Q4: Why is my DoE only 10 cases? Isn't that small?

For a quadratic response surface in 3 factors we need 10 distinct points minimum (Box–Behnken or central composite design with 1 centre point). Sliced LHS at 10 points gives the same coverage with better space-filling properties. A future objective would be to add a second wave of 10 cases at the inflection regions identified by the first wave (sequential DoE), but that is beyond the MTP scope.

Q5: Symmetry plane — aren't real T-junctions asymmetric?

The instantaneous flow is asymmetric (vortex shedding, turbulent fluctuations), but the *time-averaged* flow is mirror-symmetric under the $x \to -x$ reflection because the inlet conditions are. RANS solves the time-averaged equations directly. Half-domain RANS of T-junctions has been validated against full-domain RANS in Chen et al. (2017), Frank et al. (2010), Walker et al. (2010); the time-averaged QoIs (CoV, Δp) agree to within $\sim 1\%$. The symmetry would NOT be valid for LES.

Q6: Why rhoReactingBuoyantFoam and not rhoPimpleFoam?

Three things distinguish them:

1. `rhoReactingBuoyantFoam` solves species transport natively with full multispecies thermo (JANAF, mixture viscosity, Sutherland). `rhoPimpleFoam` only solves enthalpy and a single-species ρ.
2. It uses p_{rgh} (gravity-corrected pressure), so buoyancy is not an explicit body-force term in the momentum equation but is built into the pressure decomposition.

3. It includes a chemistry hook (we set `combustion off` and an empty `reactions` dictionary, but the solver machinery is there if we ever need it).

Q7: You report CoV at one downstream station. Why not several?

We *do* sample H_2 at four downstream stations ($z = 3, 4, 5, 6\,\mathrm{m}$) using `sampledSurface` function objects in `controlDict`. The headline CoV is at the outlet ($z = 6.9\,\mathrm{m} = 4.6\,D$ downstream of the junction) because that's the relevant "custody-transfer" station. The upstream four stations give the *mixing length* — how far downstream you'd need to go to drop CoV below a threshold.

Q8: How do you validate the mesh?

Three layers:

1. `checkMesh -allTopology -allGeometry`: max non-orthogonality 49°/65°, max skewness 2.57/4, layer coverage 99.6%. All within OpenFOAM healthy thresholds.
2. Grid-independence: 230 k, 460 k, 920 k cells. CoV changed by 0.4% (coarse→medium) and 0.2% (medium→fine). Adopted the medium for the campaign.
3. Single connected fluid region: `checkMesh` reports `Number of regions: 1 (OK)` and a Python `pyvista.connectivity` check confirms it. (Crucial after the 30° mesh-defect fix.)

Q9: What if the prof asks "why isothermal?"

Two reasons:

1. Industrial gas pipelines run essentially isothermal (5°C variation over kilometres). The Joule–Thomson coefficient of H_2 is small in our pressure regime and would only matter near regulators.
2. Disabling the energy equation (relTol 1, maxIter 0) saves $\sim 20\%$ wall time per case, which over the DoE matters. If we later cared about thermal effects we would just re-enable it.

Q10: What is the limitation of this study?

Honestly stated:

- RANS time-averaging hides the unsteady mixing structures that LES would resolve.
- Wall functions ($y^+ \sim 30$–80) sacrifice near-wall-resolution; a $y^+ \sim 1$ run would be the next refinement.
- No combustion, no chemistry. We are studying mixing only.
- Single fluid pair (H_2/CH_4). A real pipeline has multi-component natural gas (CH_4, C_2H_6, N_2, CO_2).
- One operating pressure (6.9 MPa). High-pressure CFD requires real-gas EOS, which we have not used.
- Symmetry plane assumption excludes any pitchfork instability the geometry may admit at higher VR.

Q11: Why y+ around 30-80 specifically and not just any value?

The wall-function regime is valid for $y^+ > 11$ (above the viscous sublayer) and $y^+ < 200$ (still inside the log layer). $y^+ \sim 30$–80 keeps us comfortably inside that band on every mesh and at every cell, with margin for the highest-VR case where near-wall velocity spikes increase u_τ.

Q12: How do you know the simulation is converged?

Three checks per case:

- Final residuals: $\mathbf{u} \sim 10^{-9}$, $p_{\text{rgh}} \sim 10^{-7}$, $H_2 \sim 10^{-8}$.
- Cumulative continuity error $\sim 5 \times 10^{-3}$ ($< 10^{-2}$ acceptance).
- Time-traces of outlet H_2, Δp flatten over the second flow-through (the moving average is constant to $\sim 1\%$ over the averaging window).

Q13: Why 1.2 s endTime?

Two main-pipe flow-throughs ($L_{\text{main}}/U_{\text{main}} = 6.9/10 = 0.69\,\text{s}$). The first one is the start-up transient (filling the pipe with the inlet condition), the second is the window we time-average over. This is standard practice for transient RANS of internal pipe flow (Ayach 2017).

Q14: What does "mesh-reuse from case_03" mean?

A sliced LHS pairs cases by d/D. Cases 03 and 04 both have $d/D = 0.252$, so they have the *same* geometry and *same* mesh. Building the mesh takes ~ 30 minutes (`snappyHexMesh` + layers); reusing it means just `cp -r constant/polyMesh` and the `processor*/constant/polyMesh` sub-meshes from the donor. This trick saves about 4 hours over the 10-case campaign.

Q15: What happens if a case crashes mid-run?

The hardened `Allrun` (in spirit) and the orchestrator's failure handler ensure:

- `reconstructPar -latestTime` runs whatever was written before the crash;
- `SIM_FAILED` is written with the last 200 log lines;
- the campaign moves on to the next case.

This is exactly what happened on the 30° case 04 when we manually paused it to free the queue. The partial state at $t = 0.2\,\text{s}$ is still on disk and can be resumed via the dedicated `resume_case04_30deg.sh` script.

Q16: Why is the operating pressure 6.9 MPa?

That is the pipeline operating pressure cited by Su et al. (2022) for a typical natural-gas trunk line. We match it so our results sit in the same operating point as the literature we benchmark against.

Q17: Why stop the campaign at $\alpha = 30°$ and $150°$ and not $0°$ or $180°$?

$\alpha = 0°$ would mean injecting along the main pipe axis from upstream — physically just adding the gases through the same inlet, with no T-junction. $\alpha = 180°$ would mean injecting from downstream backward against the flow, also unphysical for an injection problem. The practical industrial range is $\sim 30°$–$150°$ depending on terrain and ROW constraints.

11. Quick command-line cheatsheet

```
# 1. Source the environment (do this in every fresh shell)
source /usr/lib/openfoam/openfoam2406/etc/bashrc

# 2. Inspect a case
foamGet -case <case>            # generate boilerplate dictionary
checkMesh -allTopology          # mesh quality
checkMesh -allGeometry
foamListTimes -case <case>      # which time directories exist
postProcess -func sample        # sample fields on planes/lines

# 3. Build the mesh
blockMesh                        # background mesh from blockMeshDict
surfaceFeatureExtract            # extract eMesh feature edges from STLs
snappyHexMesh -overwrite         # cut + snap + layers in one go
checkMesh                        # always check after meshing
createPatch -overwrite           # rename / merge patches

# 4. Decompose / reconstruct
decomposePar -force              # split for parallel
reconstructPar -latestTime       # merge latest time
reconstructPar -time 0.5:1.2     # merge a range

# 5. Run the solver
mpirun -n 16 rhoReactingBuoyantFoam -parallel
foamLog log.solver               # extract residual time series

# 6. Post-process
paraFoam -touch                  # create the .foam stub for ParaView
postProcess -func 'patchAverage(name=outlet, fields=(H2 p_rgh))'

# 7. Manage the case
foamCleanCase                    # remove time dirs and processor*
wclean libso ./mySolver          # clean a custom solver
wmake libso ./mySolver           # compile a custom solver

# 8. The DoE pipeline (this project specifically)
python3 lhs_design.py --seed 42 --outdir doe/
python3 stamp_cases.py --design doe/doe_design.csv --base . --cases ../cases
./run_doe.sh                     # the orchestrator, in a tmux session
./tools/doe_progress_30deg.sh    # snapshot of remote campaign
```

Listing 19: The 25 commands that cover 95% of OpenFOAM use

12. Final pre-viva checklist

Things I should have ready to draw on a whiteboard.

1. The T-junction geometry (main pipe horizontal, branch coming in at α, label D, d, d/D, VR, HBR, the symmetry plane).
2. The 1/7-th power-law profile shape.
3. A jet-in-crossflow sketch with the counter-rotating vortex pair, label penetration depth $\propto \sqrt{J}\, d$.
4. The Richardson / momentum-flux number axes, with "stratified vs. mixed" bands marked.
5. The PIMPLE loop diagram (predictor $\rightarrow$ pressure correction $\times$ nCorrectors $\rightarrow$ outer corrector $\times$ nOuterCorrectors).
6. The case directory tree (`0/`, `constant/`, `system/` with one example file each).

Things I should know by heart.

- Solver = `rhoReactingBuoyantFoam`; turbulence = k-ω SST; thermo = JANAF + Sutherland + perfectGas.
- Domain: $D = 0.46\,\mathrm{m}$, $L_{\mathrm{main}} = 6.9\,\mathrm{m}$, $d/D \in [0.15, 0.45]$, half-domain symmetry ($\sim 460\,\mathrm{k}$ cells).
- Time: $t_{\mathrm{end}} = 1.2\,\mathrm{s}$, $\mathrm{Co}_{\mathrm{max}} = 4.5$ (90°) or 8.0 (30°), $\Delta t \sim 10\,\mu\mathrm{s}$.
- DoE: sliced LHS, seed 42, 10 cases per angle (5 slices of 2), same parameter set across angles for clean factorial comparison.
- Three angles: 90° (done), 30° (campaign in progress, mesh-fix story), 150° (planned).
- 90° headline result: best CoV = 0.066 at VR = 3.81 (case 04); worst CoV = 0.71 at VR = 0.69 (case 08).

Things to NOT bluff if asked.

- I have not done LES. RANS only. If asked about LES, say I expect it to break the symmetry assumption and that we'd need full-domain meshes.
- I have not validated against experiment, only against Su 2022 trends (which is qualitative, not absolute-number, agreement).
- The 30° and 150° campaigns are at different stages of completion; the cross-angle synthesis is the final-viva deliverable, not done yet.
- Real gas effects (Lee–Kesler, Peng–Robinson) are not modelled. Operating pressure is well below the H_2 critical point, so this is an acceptable approximation but not physics-tight.